Assignment 3: Differential Privacy and Synthetic Data

Michail Pettas | mai25mps@cs.umu.se | Data Privacy course | September 2026

1. Introduction and setup

In this assignment I made differentially private versions of the California Housing dataset with the Laplace mechanism, and synthetic versions with CTGAN, and compared how useful they are for a regression task. The dataset (`sklearn.datasets.fetch_california_housing`) has 20,640 block groups from the 1990 census, 8 numeric features (`MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude`, `Longitude`) and the target `MedHouseVal`, the median house value in units of $100,000. There are no missing values.

I split the data once, 80/20 with seed 42, into 16,512 training and 4,128 test records. The test set is never noised, never given to CTGAN and not used when fitting the pre-processing. All the DP and synthetic datasets are made from the training split only. For each of them I train a regressor and evaluate it on the real test set (often called "train on synthetic, test on real"). This measures how useful the released data would be for predicting real house prices. Predictions are converted back to $100k units, so MSE is in ($100k)² and MAE in $100k.

Both the Laplace noise and CTGAN are random, so I repeated every configuration: 5 noise draws for each DP setting and 3 CTGAN runs (different subsample and seed) for each sample size. The tables show the mean, and the shaded areas in the figures show the min and max over the runs. Two reference values are used throughout the report. The regressor trained on the original data gets MSE 0.216 and MAE 0.310, which is the best we can expect. Always predicting the training mean gives MSE 1.311 and MAE 0.906, so a dataset that does not beat this has no useful information for the task.

2. Pre-processing (Task 1)

I used Min-Max scaling to [0, 1], after first clipping (winsorising) every feature to its 1st and 99th percentile on the training split. The target is already bounded (0.15 to 5.0) so it is not clipped. The clipping bounds and the `MinMaxScaler` are fitted on the training data only, and then applied unchanged to the test set and to all synthetic data.

The main reason for choosing Min-Max is the Laplace mechanism in Task 2. The noise has to be calibrated to the sensitivity, i.e. how much one person's record can change a value. After Min-Max scaling every attribute lies in an interval of width 1, so all attributes have sensitivity 1 and the same noise scale can be used for all of them. Standardised values do not have a fixed range, so the bounds would anyway have to come from clipping, and each attribute would get a different sensitivity (its clipped range divided by its standard deviation). `RobustScaler` has the same problem.

The clipping is needed because `AveRooms`, `AveBedrms`, `Population` and `AveOccup` have very large outliers. For example, `AveOccup` has a median of 2.8 but a maximum of 1,243. With plain Min-Max scaling almost all records end up in a tiny interval near 0 (Figure 0 in the notebook), and noise calibrated to the full range would completely hide the real variation. The clipping changes about 2 % of the values of each feature, and it does not hurt the model: the MSE on the original data is 0.216 both with and without clipping.

One limitation is that the percentile bounds are computed from the private data. For a strict DP guarantee they should come from public knowledge, or be estimated with a part of the privacy budget. I treat them as public, which is a common simplification. Also, the regressor I use is tree-based and does not depend on feature scaling, so the scaling only matters for the noise and not for the model.

3. Choice of regressor (Task 4)

I compared three standard regressors on the original data (Table 1) and chose scikit-learn's `HistGradientBoostingRegressor` with default settings and early stopping (10 % of the training data used for validation). It had the lowest error. House prices depend non-linearly on location and income, e.g. there are clusters around Los Angeles and the Bay Area, and linear regression cannot model this (its MSE is about twice as high). Early stopping is useful for the noisy datasets: when the training target is mostly noise, the validation loss stops improving quickly and the model stops instead of fitting the noise. It is also fast, about 0.2 s per fit compared with about 0.7 s for a random forest, which matters because the whole study needs more than 200 fits.

Table 1. Regressors trained and tested on the original data

Model	MSE	MAE
Linear regression	0.449	0.488
Random forest (100 trees)	0.257	0.329
Histogram gradient boosting	0.217	0.310
Mean predictor (reference)	1.311	0.906

4. Differentially private datasets (Task 2)

Each scaled training record $x \in [0,1]^9$ (the 8 features and the target) is released with independent Laplace noise added to every attribute (the Laplace mechanism [1]):

$$\tilde{x}_j = x_j + Y_j, \quad Y_j \sim Lap(0, b), \quad b = \Delta_1 / \varepsilon = 9 / \varepsilon$$

If one person's record is replaced, each of the 9 attributes can change by at most 1, so the L1 sensitivity of a record is $\Delta_1 = 9$. By sequential composition this is the same as spending $\varepsilon/9$ on each attribute. Since every record is noised independently, parallel composition gives that the whole dataset is ε-DP. Strictly speaking this is local DP, since every record is protected on its own, even from whoever collects the data. Many examples instead add $\mathrm{Lap}(1/\varepsilon)$ to each attribute, but that spends ε per attribute, so a record is only 9ε-DP. My ε therefore corresponds to $\varepsilon/9$ in that convention (column 4 of Table 2). I did not clip the noisy values back to [0, 1]. It would be allowed as post-processing, but with a lot of noise it pulls the mean of every attribute towards 0.5, while the unclipped noise has mean zero.

Values of ε: 0.1, 0.5, 1, 5, 10, 25, 50, 100, 250, 500, 1000 and 5000. I used such a wide, log-spaced range because local DP needs a large ε before a single noisy value says anything: the noise standard deviation $\sqrt{2}\cdot 9/\varepsilon$ is larger than the whole [0, 1] range of an attribute for all ε below 12.7. The grid covers strong privacy ($\varepsilon \le 1$), the range where utility recovers (10–500) and almost no privacy (1000–5000).

Table 2. Noise level and utility of the DP datasets (mean of 5 noise draws)

ε	Laplace scale b	noise std	ε/9 per attribute	MSE	MAE	MSE / original
0.1	90	127.3	0.011	218.3	12.31	1012
0.5	18	25.5	0.056	3.552	1.492	16.5
1	9	12.7	0.111	3.432	1.525	15.9
5	1.8	2.55	0.556	1.405	0.933	6.5
10	0.9	1.27	1.11	1.229	0.869	5.7
25	0.36	0.509	2.78	0.975	0.764	4.5
50	0.18	0.255	5.56	0.638	0.611	3.0
100	0.09	0.127	11.1	0.432	0.479	2.0
250	0.036	0.051	27.8	0.318	0.391	1.47
500	0.018	0.025	55.6	0.280	0.360	1.30
1000	0.009	0.013	111	0.253	0.339	1.17
5000	0.0018	0.0025	556	0.223	0.316	1.03

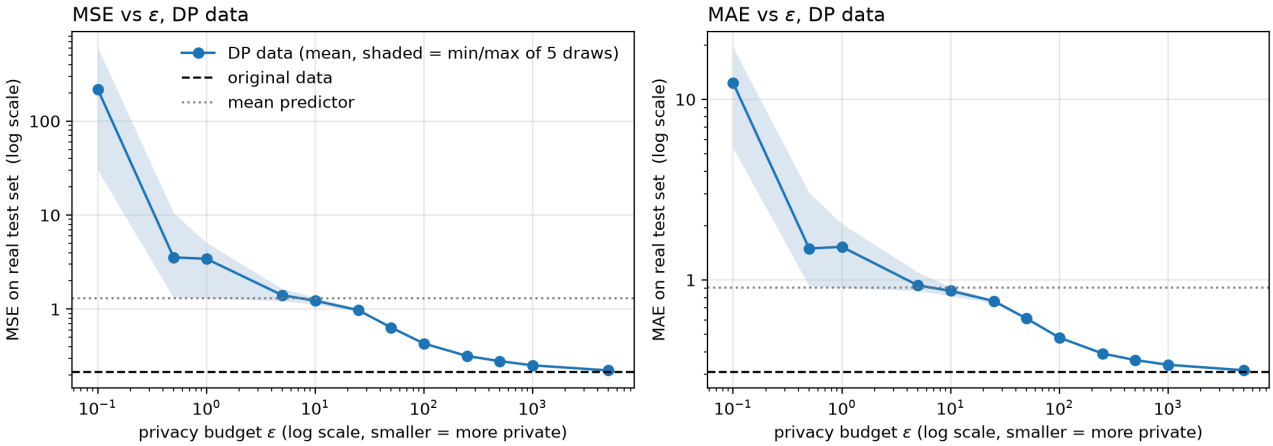


Figure 1. Utility of the DP datasets: MSE (left) and MAE (right) on the real test set against ε, both axes in log scale. The line is the mean of 5 noise draws and the shaded area the min/max. The dashed line is the original data and the dotted line the mean predictor.

5. Synthetic datasets (Task 3)

For the synthetic data I used the `CTGANSynthesizer` from SDV [3] (version 1.38) with the default settings of 300 epochs and batch size 500. CTGAN [2] is a GAN designed for tabular data. It uses mode-specific normalisation, where each continuous column is modelled with a Gaussian mixture, which suits the skewed columns here (`Population`, `AveOccup`) and the latitude/longitude distribution with its two big city clusters. The generator also learns the joint distribution of all columns, so in principle it can keep the relation between the features and the target, which is what the regressor needs. `GaussianCopulaSynthesizer` would be much faster but only models correlations, and `TVAESynthesizer` often performs about as well as CTGAN; I used CTGAN since the assignment suggests it.

CTGAN is trained on the raw training records because it normalises the data itself. Its output is then clipped and scaled in the same way as the real data. One problem I had to fix was that SDV's automatic

metadata detection marks `Latitude` and `Longitude` as personal information and replaces them with random coordinates in the generated data. Since location is the most important predictor of house value, I changed both columns back to `numerical`.

Number of original samples used: n = 10, 100, 1,000, 5,000 and 16,512 (the full training set). For each n I ran CTGAN 3 times with a different random subsample and seed, which gives 15 synthetic datasets. Every synthetic dataset has 16,512 rows, the same as the real training set, so the only difference between them is how much real data CTGAN was trained on.

Besides utility, Table 3 shows the SDV quality score (how similar the synthetic data is to the real training data, 0–1) and a simple privacy check, the distance to closest record (DCR). The DCR ratio is the median distance from the synthetic rows to CTGAN's training rows, divided by the same median for real rows that CTGAN did not see. A value well below 1 would suggest that CTGAN copies its training records.

Table 3. CTGAN synthetic datasets (mean over 3 runs, ± standard deviation for MSE)

n	MSE	MAE	MSE / original	SDV quality	DCR ratio	exact copies	fit time
10	1.469 ± 0.290	0.936	6.8	0.66	1.08	0 %	6 s
100	1.836 ± 0.457	1.075	8.5	0.67	1.91	0 %	6 s
1,000	1.708 ± 0.414	0.948	7.9	0.68	2.30	0 %	21 s
5,000	1.260 ± 0.024	0.876	5.8	0.76	2.09	0 %	90 s
16,512	0.767 ± 0.002	0.678	3.6	0.86	1.50	0 %	260 s

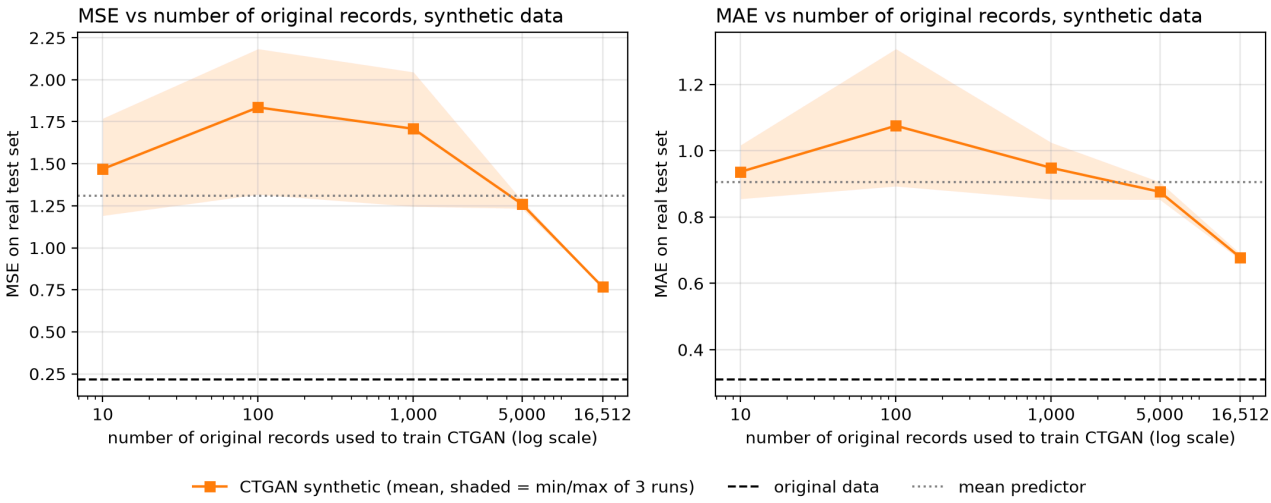


Figure 2. Utility of the synthetic datasets: MSE (left) and MAE (right) on the real test set against the number of original records CTGAN was trained on (log scale). The line is the mean of 3 runs and the shaded area the min/max.

6. Utility compared with the original data (Task 4)

Differentially private data

For ε ≤ 5 the DP data is useless. The MSE is at or above the mean predictor, and at ε = 0.1 it is 218 with an MAE of 12.3, on a target that only goes from 0.15 to 5. At that ε the noise on the target has a standard deviation of 127 in scaled units, so even the average of the noisy targets is off by around 5 ($100k), and the trees also fit some of the noise. The predictions end up far outside the valid range, which is why the

model is much worse than just predicting a constant. The shaded area in Figure 1 is also very wide for $\varepsilon \leq 1$, so the result depends a lot on the particular noise draw.

Between $\varepsilon = 10$ and 500 the MSE goes down smoothly from 5.7 to 1.3 times the original, and the variation between draws gets smaller. The noise on the target has mean zero and averages out over 16,512 records, which is why a useful model can be learnt even though every single value is quite noisy. The noise on the features is a bigger problem, because it blurs the relation between input and output (errors-in-variables), and the model cannot undo that. Most of the remaining gap probably comes from this. For $\varepsilon \geq 1000$ the data is almost as good as the original (1.17 times the MSE at 1000 and 1.03 times at 5000), but such large values of ε give very little real protection.

So with record-level Laplace noise, any ε that gives a meaningful guarantee (roughly $\varepsilon \leq 5$) destroys the data. This is mostly because it is local DP, where every value has to be protected individually. Central DP methods, for example training the model with DP-SGD or using a DP synthesizer, would need much less noise for the same ε.

Synthetic data

With $n \leq 1{,}000$ the synthetic data does not contain useful information: the mean MSE (1.47–1.84) is above the mean predictor, and the runs vary a lot (from 1.19 to 2.18). The SDV quality score of 0.66–0.68 shows that the rough column distributions are reproduced, but the relation between the features and the target is not. Part of the explanation is that with a fixed 300 epochs and batch size 500, CTGAN only does 300–600 gradient steps for these sample sizes, compared with 9,900 for the full training set. The order between n = 10, 100 and 1,000 is within the run-to-run variation, so I would not read it as a trend.

At n = 5,000 the MSE (1.26) is only slightly better than the mean predictor, but the three runs are now very similar, and the quality score increases to 0.76. With all 16,512 records the MSE is 0.767 and the MAE 0.678, stable across runs, with quality 0.86. This model is clearly useful, with 41 % lower MSE than the mean predictor, but it is still 3.6 times the original MSE (2.2 times the MAE). My interpretation is that CTGAN reproduces the individual columns well but smooths out the finer interactions between location and income that gradient boosting uses on the real data.

The assignment assumes that synthetic data gives utility comparable to the original. In my results this is only true in a loose sense: the best synthetic data is informative, but much worse than the original. Training longer, tuning CTGAN or trying `TVAESynthesizer` might reduce the gap.

Regarding privacy, none of the synthetic rows is an exact copy of a training row, and all DCR ratios are at least 1, so the synthetic records are not closer to CTGAN's training data than new real records are. The lowest ratio is for n = 10 (1.08). With only 10 source records and SDV's default min/max enforcement, all synthetic rows lie inside the range spanned by those 10 records, which in itself reveals something about them. These are only empirical checks, and CTGAN does not give any formal privacy guarantee.

7. Matching ε for the synthetic data (Task 5)

To find the ε at which the DP data has the same utility as a synthetic dataset, I interpolated the mean DP curve (MSE against log ε, first made non-increasing) at each synthetic MSE. I then checked each matching ε^* by generating 5 new DP datasets at that ε and scoring them.

Table 4. ε at which DP data has the same MSE as each synthetic dataset

CTGAN trained on n	synthetic MSE	matching ε*	ε*/9 per attribute	DP MSE at ε* (check)
10	1.469	4.8	0.53	1.560 ± 0.278
100	1.836	3.6	0.39	1.396 ± 0.100
1,000	1.708	3.9	0.44	1.419 ± 0.058
5,000	1.260	8.9	0.98	1.291 ± 0.031
16,512	0.767	38.4	4.26	0.780 ± 0.017

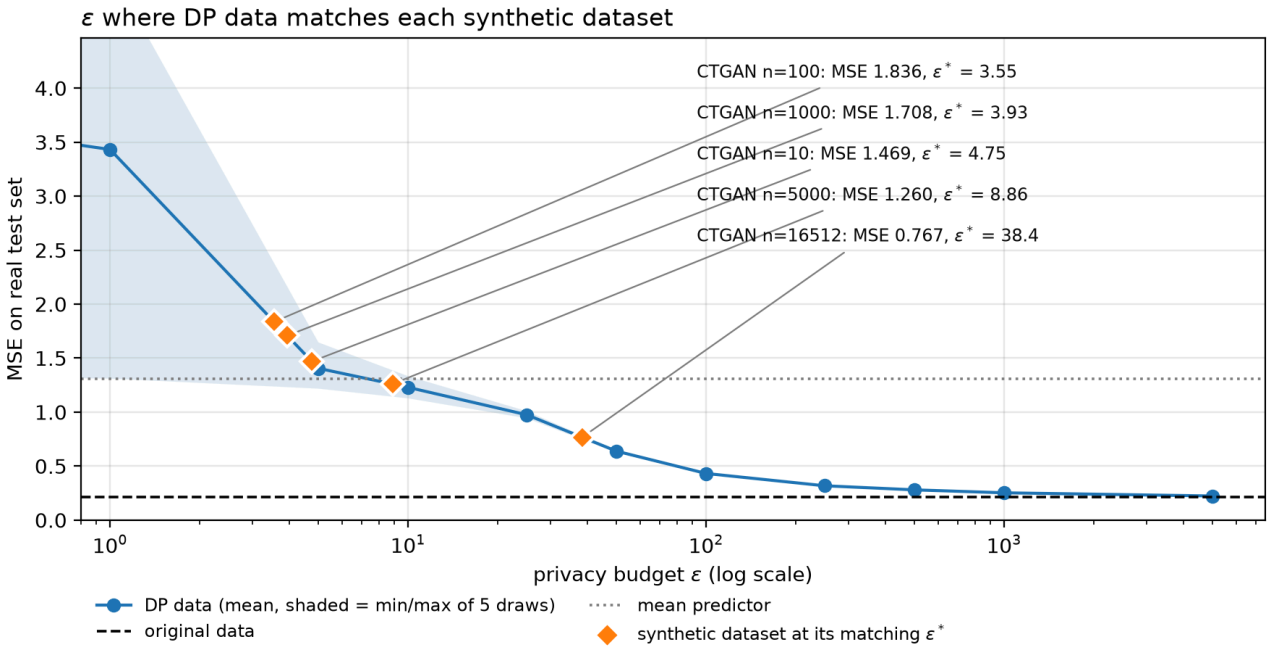


Figure 3. Each synthetic dataset placed on the DP utility curve at its matching ε* (linear y-axis, only ε ≥ 1 shown).

The best synthetic dataset (CTGAN trained on all records) matches the DP data at ε ≈ 38, and the check agrees well (0.780 against 0.767, a difference of 1.7 %). The n = 5,000 datasets match ε ≈ 9 (1.29 against 1.26). For n ≤ 1,000 the interpolation gives ε* around 4–5, but here both kinds of data are at or above the mean predictor and the DP results vary a lot between draws, so the checks are much less accurate. All these matches really say is that both datasets are uninformative, and the exact ε* values should not be taken seriously.

It is worth noting that the two approaches do not give the same kind of protection. At ε ≈ 38 the Laplace data still has a formal (although weak) DP guarantee, while the CTGAN data gives the same utility without any formal guarantee, only the empirical DCR results. So for the same utility I would prefer the DP data in this case. The problem with it is that small, meaningful values of ε give no utility at all.

8. DP applied to the synthetic data (Task 6)

I took the three synthetic datasets trained on all records, scaled them as before, and applied the same Laplace mechanism with **ε = 1, 10, 50, 100, 250, 500, 1000 and 5000**, with 5 noise draws each (15 models per ε). The regressor was again evaluated on the real test set.

Table 5. Utility of the DP synthetic data (synthetic data without noise: MSE 0.767)

ε	MSE	MAE	MSE change vs synthetic	DP on original, MSE
1	2.029	1.110	+164 %	3.432
10	1.257	0.892	+64 %	1.229
50	0.940	0.779	+23 %	0.638
100	0.814	0.721	+6.1 %	0.432
250	0.752	0.682	−1.9 %	0.318
500	0.745	0.673	−2.8 %	0.280
1000	0.756	0.676	−1.4 %	0.253
5000	0.764	0.677	−0.4 %	0.223

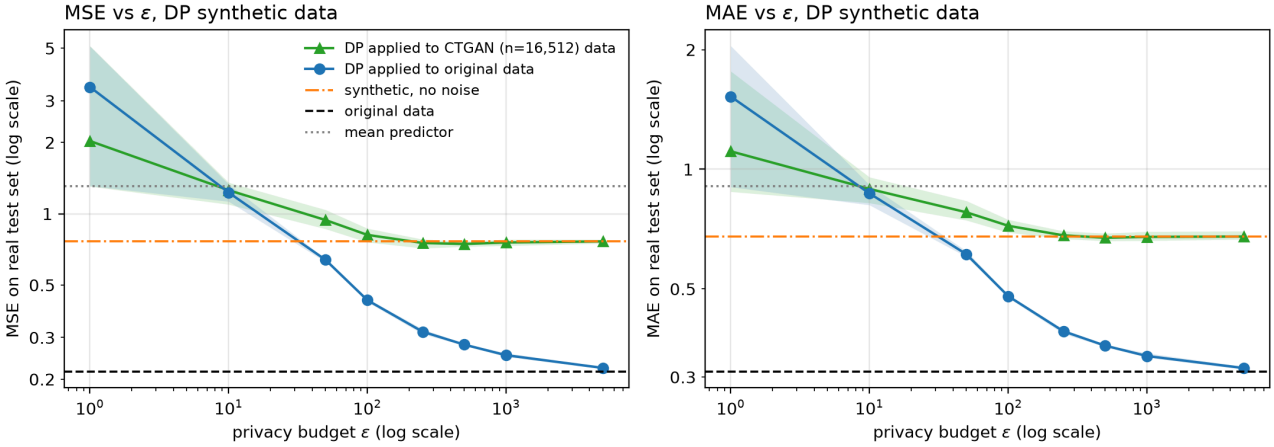


Figure 4. Utility of the DP synthetic data: MSE (left) and MAE (right) against ε, both axes in log scale. Green: DP applied to the CTGAN data (mean of 15 models, shaded min/max). Blue: DP applied to the original data, for comparison. The orange dash-dot line is the synthetic data without noise.

The noise reduces the utility up to about ε = 100. The MSE increases by 164 % at ε = 1, by 64 % at ε = 10 (where it is about the same as the mean predictor), by 23 % at ε = 50 and by 6 % at ε = 100. From ε = 250 the change is within ±3 %. The small negative changes are within the variation between runs (or possibly a small regularising effect of the noise), so they should not be seen as an improvement. Relative to its own starting point, the synthetic data loses less utility than the original data at the same ε. The noise is equally large in both cases, but the synthetic data has already lost much of the fine structure that the noise would otherwise destroy.

In absolute terms, adding the noise to the original data gives a lower error than adding it to the synthetic data for every ε ≥ 10, and below that both are useless. It is also important that the noise only makes the release ε-DP with respect to the synthetic records. CTGAN was trained on the real data without DP, so anything it has memorised is passed on, and the whole pipeline is not ε-DP for the real people in the data. To get a real guarantee the synthesizer itself has to be trained with DP, e.g. DP-CTGAN or PATE-CTGAN from SmartNoise, or CTGAN trained with DP-SGD using Opacus. The generated data would then be DP by the post-processing property, without adding extra noise.

9. Conclusion

The privacy-utility trade-off of the Laplace mechanism is steep in this setting: values of ε that give meaningful record-level privacy (around 5 or lower) leave no utility, and utility close to the original needs $\varepsilon \geq 1000$. The utility of the CTGAN data depends strongly on how much real data it is trained on. It is only useful when trained on all 16,512 records, and then it is comparable to Laplace DP at $\varepsilon \approx 38$ (3.6 times the original MSE). Adding Laplace noise to the synthetic data reduces its utility for $\varepsilon \leq 100$ and does not protect the original individuals, so a DP synthesizer would be the right tool for private synthetic data.

The main limitations are that the clipping bounds come from the data, that CTGAN uses a fixed number of epochs so the small samples also get less training, that only one regressor is used, and that the privacy of the synthetic data is only checked empirically with DCR and not with, for example, membership inference attacks.

All code is in `Assignment3_DP_Synthetic.ipynb` with fixed seeds, and package versions are listed in `requirements.txt`. The synthetic datasets are saved in `outputs/synthetic/`, so re-running the notebook takes about a minute instead of about 25 minutes.

References

[1] C. Dwork and A. Roth. The Algorithmic Foundations of Differential Privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4), 2014.

[2] L. Xu, M. Skoularidou, A. Cuesta-Infante and K. Veeramachaneni. Modeling Tabular Data using Conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[3] N. Patki, R. Wedge and K. Veeramachaneni. The Synthetic Data Vault. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016.